

Big Data Analytics & Cloud Computing
Project Report



**Automated Damage Assessment System for Natural
Disasters**

Submitted By
Wai Phu Paing (200530777)
Shubham M. Gandhi (200526729)

Submitted To
Dr. Lisa Fan

August 16, 2025

Abstract

Natural disasters cause billions of dollars in economic damage each year and place a heavy burden on humanitarian response systems. Traditional damage assessment methods—based on manual inspections or isolated image analysis models—are slow, fragmented, and not scalable for large-scale events.

This project research integrates data from multiple sources to enable automated damage classification and risk prediction by using a cloud-native, AI-driven disaster assessment framework deployed on Microsoft Azure.

An ensemble comprising ResNet-50, DenseNet-121, and EfficientNet-B0 is trained on the xBD satellite imagery dataset to classify building damage into four distinct levels. Outputs from the building damage classification are supplemented with meteorological, demographic, and geospatial variables from NASA POWER, WorldPop, and OpenCage. Using these combined features, an XGBoost risk model generates a four-level disaster risk assessment that captures both structural impact and population vulnerability.

Azure Functions manage automation by running pipeline orchestration and hosting the Azure Kubernetes Service endpoint as a real-world integration. In testing, 67.5% accuracy was obtained for damage classification and 70.1% for risk prediction, outperforming baseline single-model methods.

The pipeline is managed within Azure Machine Learning, automated using Azure Functions, and deployed on Azure Kubernetes Service to enable real-time accessibility. Interactive Power BI dashboards, REST APIs, and a public Streamlit application provide decision-makers with accessible risk insights.

This study presents a disaster management framework that is both scalable and operational and allows for the integration of multi-source data and converts it into actionable insights within a short time frame. By taking this action, it supports faster decision-making and enables data-driven humanitarian response efforts.

Table of Contents

1	INTRODUCTION.....	1
2	PROBLEM STATEMENT	1
3	APPROACH.....	2
3.1	Introduction of the Dataset.....	2
3.2	High-Level Project Workflow.....	2
3.3	Azure Architecture for Cloud Infrastructure and Workflow Design	4
3.3.1	Azure Blob Storage — Centralized Data Repository.....	4
3.3.2	Azure Machine Learning — Orchestration of Analytical Pipelines.....	4
3.3.3	Azure Functions — Event-Driven Automation Layer.....	10
3.3.4	Container Registry (ACR) — Model Image Repository.....	10
3.3.5	Azure Kubernetes Service (AKS) — Scalable Model Serving.....	10
4	RESULTS	11
4.1	Model Performance.....	11
4.1.1	Preprocessing Results.....	11
4.1.2	Damage Classification Model Results.....	12
4.1.3	Risk Prediction Model Results	13
4.2	Visualization Output	15
4.2.1	Power BI Dashboard.....	15
4.2.2	API Endpoints (AKS Deployment)	17
4.2.3	Streamlit Public Application	17
5	DISCUSSION OF RELATED WORK AND COMPARISON.....	19
6	LIMITATIONS AND POSSIBLE EXTENSIONS	19
7	CONCLUSIONS.....	20
8	REFERENCE.....	21
9	APPENDIX	22
9.1	Appendix A — Sample Output Screenshots for Streamlit Public App	22
9.2	Appendix B — Sample Output Screenshots for EndPoint	24
9.3	Appendix C — Implementation, Coding, and Results Documentation	25

1 INTRODUCTION

Natural disasters cause over 268 billion dollars in global damages annually, often worsened by delays in damage assessment that hinder emergency response and result in inefficient resource use [1]. According to the United Nations Office for the Coordination of Humanitarian Affairs (UN OCHA), only 43% of the needed humanitarian funding was provided in 2023, leaving a historic funding shortfall of USD 35 billion [2], which directly affects the ability to deliver timely and adequate relief. Current assessment workflows depend on manual inspections or single-source remote sensing models, which are slow, fragmented, and cannot scale for large-scale events involving hundreds of thousands of images.

This research is driven by the 2025 Myanmar earthquake, a 7.7 magnitude disaster that devastated several cities in the author's home country, Myanmar. Because of the country's political and logistical challenges, disaster response teams were unable to access affected regions promptly, leading to considerable loss of life and families stranded under collapsed buildings for several days. The tragedy illustrated the life-saving potential of rapid, automated assessments; however, the experimental evaluation in this study is based on historical disasters from the xBD dataset rather than direct Myanmar earthquake data [12].

To address the challenges of delay, data fragmentation, and limited scalability, this work proposes an AI-driven, cloud-native disaster damage assessment, and risk prediction system deployed entirely on Microsoft Azure.

The system integrates:

1. Damage classification using an ensemble of ResNet-50, DenseNet-121, and EfficientNet-B0 trained on the xBD dataset with class balancing and extensive augmentation.
2. Risk prediction using an XGBoost classifier combining predicted damage levels with weather (temperature, wind speed, rainfall) and population density data.
3. Automated orchestration via Azure Machine Learning pipelines and event-triggered execution through Azure Functions, and
4. Scalable delivery through Azure Kubernetes Service APIs and interactive Power BI/Streamlit dashboards.

The primary objective is to enable "right aid to the right place at the right time" by transforming multi-source raw data into actionable intelligence within hours, rather than days or weeks.

2 PROBLEM STATEMENT

Disaster damage assessment is among the most resource-demanding and urgent aspects of humanitarian response. Conventional methods largely rely on manual inspections or standalone computer vision models that analyze satellite imagery without extra context. While manual assessments provide precise insights, they are not feasible for large-scale disasters involving thousands of buildings and communities simultaneously. Automated single-source models, despite being scalable, often lack robustness and contextual awareness, leading to misclassification of damage levels and poor prioritization of the most at-risk areas [13]. These challenges result in delays in situational awareness, inefficient resource deployment, and ultimately, unnecessary loss of life.

A significant challenge in disaster analytics is the fragmented data landscape and the lack of a unified analysis pipeline. Previous research mainly focused on damage detection through images, but essential elements like weather patterns and demographic data are frequently ignored. Ignoring these factors results in an incomplete assessment of risk, since both structural damage and human exposure—such as populations and vital assets—are not thoroughly represented. Furthermore, many solutions introduced in the literature exist only as experimental models, with little attention to operational deployment, which limits their usefulness during real-time emergency response.

Addressing this issue, the present work focuses on the design of an integrated disaster assessment system that:

1. performs building damage classification using satellite imagery,
2. incorporates weather and demographic indicators to estimate post-disaster risk, and
3. Supports cloud-native deployment with automated dashboards and APIs.
4. Overcoming this challenge ensures aid reaches the right places quickly, shifting disaster response from reactive to proactive.

3 APPROACH

3.1 Introduction of the Dataset

1. xBD Dataset — The xBD dataset [3] comprises post-disaster satellite imagery in .png format accompanied by metadata in .json format. It has a total size of 52 GB and spans the period from 2011 to 2019. The dataset contains six disaster types—Earthquake, Wildfire, Flooding, Tsunami, Volcano, and Wind—across eight different countries.
2. OpenCage API — The OpenCage Geocoding API [4] provides location-based metadata in JSON format. Data retrieval is parameterized by an API key, latitude, and longitude, returning country name, ISO country code, and region information.
3. NASA POWER API — The NASA Prediction of Worldwide Energy Resources (POWER) API [5] supplies meteorological data in JSON format. The parameters include start date, end date, latitude, longitude, and community type. Returned variables include near-surface air temperature, total corrected precipitation, and wind speed.
4. WorldPop API — The WorldPop API [6] delivers high-resolution population density data in Geotiff format. Parameters include start date, end date, latitude, longitude, and community type. The retrieved data are used to quantify human exposure in disaster-affected regions.

3.2 High-Level Project Workflow

The proposed system uses a modular, AI-driven pipeline for disaster damage assessment, combining satellite images with weather and population data. The workflow is shown in Figure 1 as follows:

Step 1: Satellite images from the xBD dataset undergo pre-processing to extract visual features and are then categorized into damage levels (0–3). An ensemble

deep learning scheme—built on ResNet-50, DenseNet, and ElasticNet—handles the classification. This design leverages the representational strength of convolutional neural networks together with the regularization capacity of linear models, improving overall robustness.

Step 2: The damage predictions are further augmented with live weather information and population distribution metrics. This integration introduces environmental and demographic context, enriching the assessment beyond visual analysis alone.

Step 3: The fused dataset is passed into a risk estimation module developed using the XGBoost algorithm. This component assigns risk scores (0–4) by simultaneously considering structural damage intensity and contextual vulnerability factors, thereby supporting more accurate prioritization of emergency operations.

Step 4: Final outputs are delivered through interactive visualization interfaces, including a Streamlit application , API, and a Power BI dashboard with damage heatmaps, resource allocation maps, prioritized response lists, and real-time weather overlays, enabling emergency teams to make rapid, data-driven decisions during disaster events.

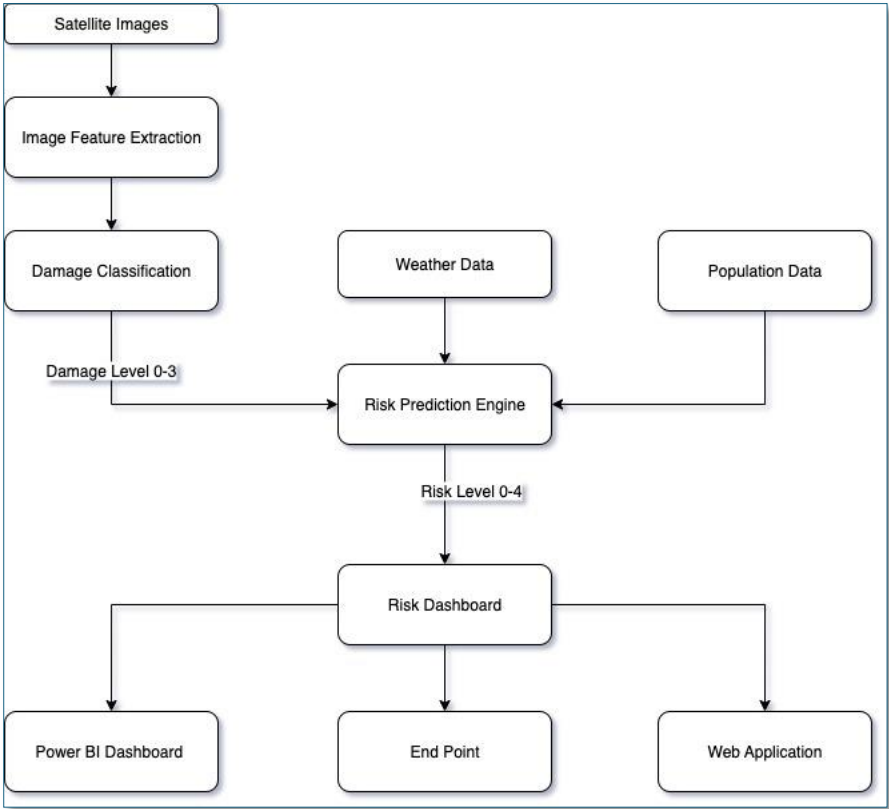


Figure 1: High-level workflow for disaster risk assessment for emergency response.

3.3 Azure Architecture for Cloud Infrastructure and Workflow Design

The proposed architecture is fully implemented on the Microsoft Azure cloud platform to leverage its elastic computing, secure storage, and managed orchestration functionalities. The system integrates five principal Azure services — Blob Storage, Machine Learning, Functions, Container Registry, and Kubernetes Service — configured in a modular yet integrated manner to facilitate an automated disaster damage assessment and risk management system.

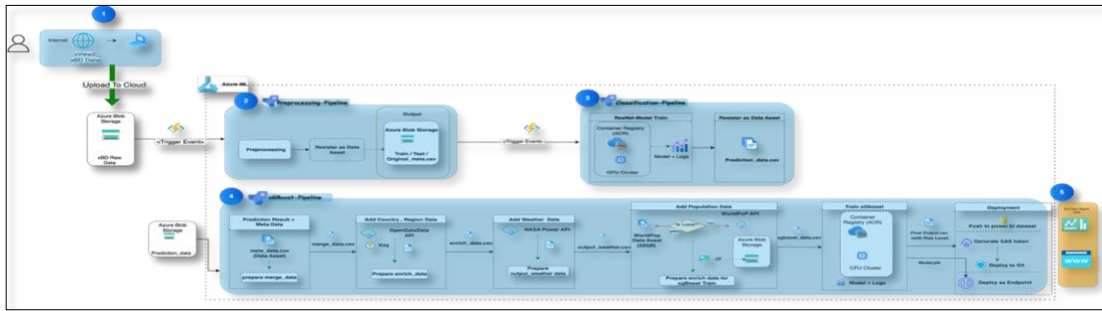


Figure 2: Proposed Microsoft Azure Workflow for Disaster Damage Risk Modelling and Scalable Deployment.

3.3.1 Azure Blob Storage — Centralized Data Repository

Azure Blob Storage serves as the primary object storage system for all data modalities employed in the pipeline. It is configured with hot-tier containers designated for frequently accessed raw datasets, and cool-tier containers for archived intermediate outputs.

The key specifications include:

1. Data Type: xView2/xBD optical satellite imagery, associated geospatial metadata, and supplementary tabular datasets.
2. Replication Policy: Geo-redundant storage (GRS) to guarantee availability in the event of regional data center failures.
3. Event Subscription: Integrated with Azure Event Grid to disseminate object creation events to Azure Functions.
4. Security: Access is restricted through Shared Access Signatures (SAS) and enforced using Azure RBAC policies [14].
5. Role in Workflow: When new or updated data is uploaded, Blob Storage emits an event that initiates downstream preprocessing in Azure Machine Learning. This architecture ensures the pipeline operates without manual intervention. [15].

3.3.2 Azure Machine Learning — Orchestration of Analytical Pipelines

Azure Machine Learning (Azure ML) functions as the fundamental computation and orchestration platform for all model development, training, and inference workflows. Its architecture utilizes Azure ML Pipelines to ensure a distinct separation of processing stages while enabling reproducible, version-controlled execution.

3.3.2.1 Preprocessing Pipeline

The preprocessing pipeline prepares the raw xView2/xBD post-disaster images with associated annotations for downstream **damage classification** and **risk prediction** tasks. This stage guarantees that all datasets are spatially aligned, class-labeled, and enriched with metadata for reproducibility and interoperability across the system pipelines.

1. Damage Class Mapping and Extraction

Each building damage subtype — no-damage, minor-damage, major-damage, and destroyed — is mapped to a numerical class index (0–3). For each post-disaster image, the corresponding GeoJSON label file is parsed to extract the subtype attribute for all annotated building footprints. The highest damage class present in the image is assigned as its overall damage label. If no valid label file exists or no building annotations are found, the image is excluded from the classification dataset.

2. Image Standardization

Images are loaded in RGB mode and resized to 224×224 pixels to meet the input size specifications of the ResNet-50 backbone. Corrupted or unreadable files are automatically omitted. The processed images are systematically organized into class-specific directory structures (class_0 through class_3), facilitating seamless integration with supervised image classification loaders in Azure Machine Learning.

3. Metadata CSV Generation for Risk Modeling

A parallel metadata extraction procedure generates a CSV file with one record for each processed image. The metadata encompasses:

- i. Image Name (image_name) — the base filename of the image.
- ii. Disaster Event (disaster) and Disaster Type (disaster_type) — derived from the metadata block within the label file.
- iii. Capture Date (capture_date) — the timestamp indicating when the satellite image was acquired.
- iv. Damage Class (damage_class) — determined as the most prevalent subtype of building damage within the image.
- v. Geospatial Coordinates (lat, lon) — the centroid latitude and longitude, obtained from the first valid building polygon in the label file's WKT geometry, utilizing Shapely for parsing.
- vi. Data Source (source) — the origin tag of the dataset (e.g., "train", "test").

4. Error Handling and Data Validation

Missing label files assigned with default values (damage_class = -1, lat/lon = -1) and are documented for traceability purposes. Invalid geometries are omitted, with the subsequent polygon being utilized if available. The completeness of the dataset is confirmed by enumerating available images and their respective labels before exporting the data to CSV.

5. Outputs

The preprocessing generates a cleaned, class-organized image dataset and a metadata-enriched CSV for downstream pipelines. The registration of these outputs as Azure Machine Learning Data Assets, ensuring dataset version control and reproducibility, is demonstrated in **Section 4.1.1**.

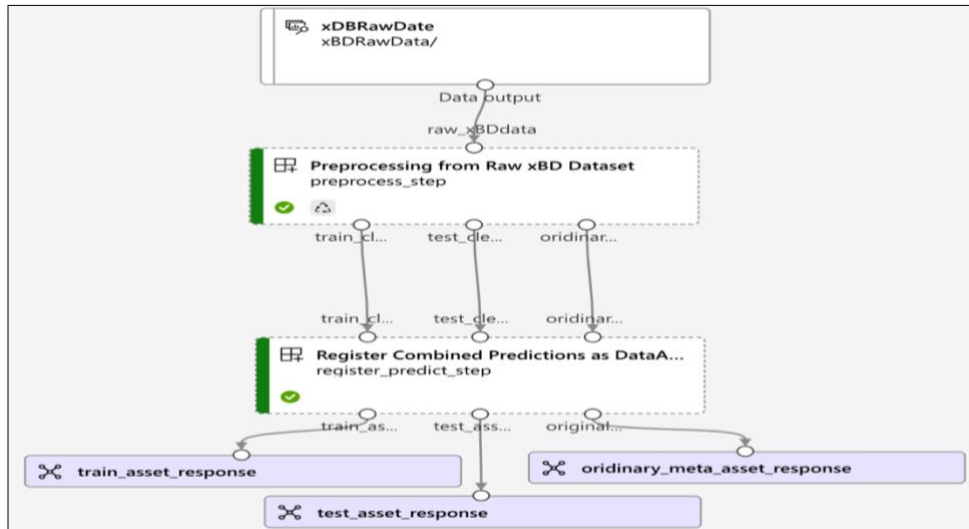


Figure 3: Azure ML End-to-End Preprocessing Pipeline

This preprocessing design ensures consistent data formatting, robust error handling, and reproducibility, forming the foundation for the automated workflows described in the subsequent subsections.

3.3.2.2 Classification Pipeline

The classification pipeline is engineered to forecast damage severity at the building level utilizing post-disaster satellite imagery. It is developed using PyTorch with integration into Azure Machine Learning for the purpose of experiment tracking, logging, and artifact management storage.

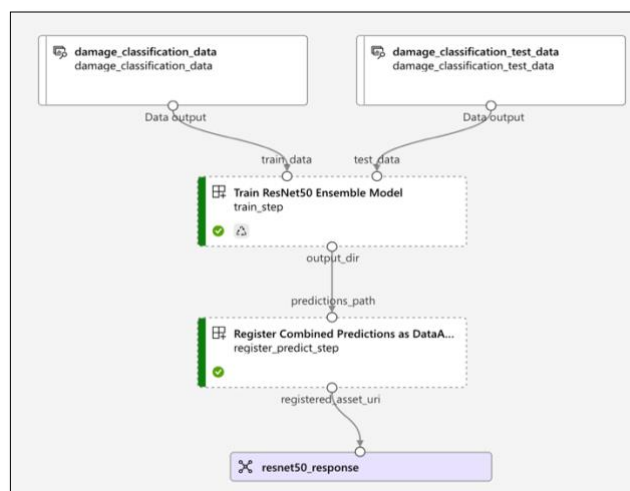


Figure 4: Azure ML End-to-End Classification Pipeline

1. Data Balancing and Augmentation

A `BalancedDataset` wrapper has been implemented to address class imbalance by oversampling minority classes to equalize their sizes with the largest class. Data augmentation techniques are employed using `Albumentations`, encompassing random horizontal flipping, geometric transformations (shift, scale, rotation), brightness and contrast adjustments, hue and saturation modifications, as well as coarse dropout masking. For validation and test sets, only resizing and normalization are utilized to ensure unbiased evaluation.

2. Dataset Partitioning

The oversampled dataset is randomly partitioned into training (80%) and validation (20%) subsets. A separate test dataset, maintained in a distinct directory, is loaded utilizing `torchvision datasets ImageFolder`. All images are resized to 224×224 pixels and normalized according to the mean and standard deviation of ImageNet deviation.

3. Model Architectures

An ensemble of three pretrained convolutional neural networks is used:

- ResNet-50 (ImageNet weights, final block unfrozen, custom fully connected layer with dropout).
- EfficientNet-B0 (final feature block unfrozen, custom classifier head).
- DenseNet-121 (final convolutional block unfrozen, custom classifier head).

All models are finely adjusted with the initial layers frozen to maintain pre-trained feature representations, while the concluding layers are adapted for a four-class damage severity classification (No Damage, Minor Damage, Major Damage, Destroyed).

4. Loss Function and Optimization

The pipeline uses weighted cross-entropy loss to address residual class imbalance after oversampling. Class weights are computed from the training set distribution. Models are optimized using the Adam optimizer with a `OneCycleLR` learning rate schedule, which cyclically adjusts the learning rate for faster convergence and reduced overfitting.

5. Training Strategy

Models are trained for a fixed number of epochs (default: 20), with early stopping initiated after five consecutive epochs without improvement in validation loss. The optimal model weights, determined by validation loss, are preserved for each architecture.

6. Ensemble Prediction

During inference, predictions from all three models are averaged at the logit level before applying the softmax activation to generate class probabilities. The class with the highest probability is assigned as the prediction.

7. Azure Machine Learning Pipeline Orchestration

The classification process is deployed and executed as an Azure ML Command Job pipeline.

- **Componentization:** The training logic is encapsulated in a reusable YAML-defined component (`train_ensemble_component.yaml`), while a separate component (`register_asset_component.yaml`) manages the registration of prediction outputs as Azure ML Data Assets.
- **Compute Resources:** A GPU-enabled cluster (`STANDARD_NC4as_T4_v3`) is automatically provisioned when the model training is started run.
- **Pipeline Definition:** The `@pipeline` decorator links the training step (and the optional asset registration step) into a reproducible workflow.
- **Execution:** The pipeline is submitted via the Azure ML Python SDK, facilitating centralized experiment tracking, output registration, and storage of model artifacts.

8. Performance Evaluation and Logging

The evaluation module computes accuracy, precision, recall, and F1-score at both per-class and macro/weighted average levels, along with confusion matrices for validation and test sets. Results are logged to Azure ML Run History, with confusion matrices, classification reports, and per-class accuracy uploaded as artifacts. Final predictions from training and test sets are saved to CSV for further analysis.

9. Output

The classification pipeline outputs a prediction CSV, performance metrics, and confusion matrices. The quantitative results and analysis for this pipeline are presented in **Section 4.1.2** and **Section 9.3 (Figure C.1)**.

3.3.2.3 XGBoost Risk Modeling Pipeline

The proposed risk prediction system utilizes an XGBoost-based multi-class classifier to estimate post-disaster risk levels by integrating geospatial, environmental, and population exposure data. The pipeline follows a modular design, deployed on Azure Machine Learning, with each step implemented as a reusable component.

1. Metadata Merge

The registered prediction CSV was combined with the original image metadata to align damage classification results with geospatial coordinates (latitude, longitude), disaster type, and capture date.

2. Data Acquisition and Enrichment

To improve the predictive capability of the model, supplementary features are derived from three external data sources:

- i. **OpenCage Geocoding API** – facilitates the conversion of latitude-longitude coordinates into standardized country codes, specifically ISO

3166-1 alpha-3, as well as regional names. This process guarantees geographic consistency across all samples and permits aggregation at the regional level.

- ii. NASA POWER API – Provides satellite-derived daily meteorological data, including near-surface air temperature (T2M), wind speed (WS2M), and rainfall, for the relevant disaster date range. These variables capture environmental drivers of disaster impact.
- iii. WorldPop Population Density Data – Supplies high-resolution raster-based estimates of population density for the corresponding country and year. These values are geographically mapped to disaster locations to measure human exposure.

3. Feature Engineering

The enriched dataset goes through preprocessing, which includes normalizing continuous features, encoding categorical variables, and calculating composite indicators like risk_score (a weighted combination of damage level, population density, and environmental variables) and categorical risk_level labels (“Low”, “Medium”, “High”, “Critical”).

4. Model Training

The enriched dataset was used to train an XGBoost classifier with the ‘multi:softprob’ objective for four-class classification: Low, Medium, High, and Critical.

The target risk level was derived from a composite risk score calculated as:

$$R = 0.4D + 0.3P + 0.2I + 0.05R_f + 0.05W$$

Where:

D= damage level , *P*= population density , *I*= infrastructure value ,
Rf= rainfall , *W* = wind speed

5. Evaluation

Model performance is evaluated utilizing accuracy, per-class precision, recall, F1-score, and confusion matrices. Training curves are generated to monitor overfitting and convergence trends. The quantitative results and analysis for this pipeline are presented in **Section 4.1.3**.

6. Output Generation

The trained model produces the output: best model file, label encoder file, and a Power BI-compatible dataset comprising predictions, probabilities, and priority scores for visualization.

7. Deployment

The final model is deployed on Azure Kubernetes Service (AKS) for real-time inference. Concurrently, the definitive risk assessment dataset is made accessible through Power BI dashboards, which are automatically refreshed via a data source API. Additionally, the dataset is published to GitHub using a Shared Access Signature (SAS) token—offering time-limited

and permission-specific access—thereby ensuring that decision-makers have the most recent risk insights.

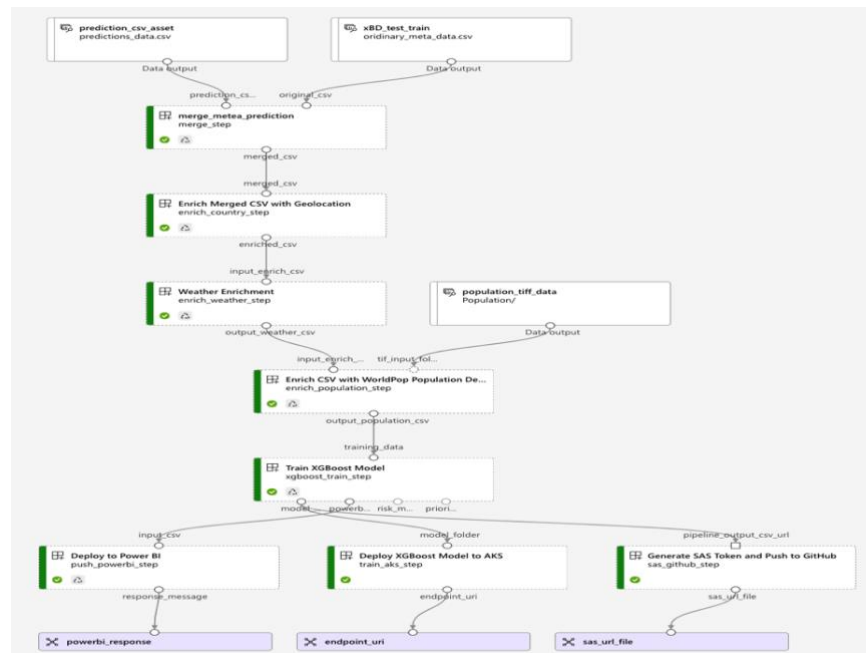


Figure 5: Azure ML End-to-End XGBoost Risk Prediction Pipeline

3.3.3 Azure Functions — Event-Driven Automation Layer

Azure Functions facilitate serverless automation, thereby eliminating the necessity for manual intervention in the initiation of pipelines.

- **Trigger Mechanism:** Monitors Blob Storage events and activates the appropriate Azure ML pipelines upon the detection of new or modified files.
- **Cost Efficiency:** Functions utilize compute resources solely during execution.
- **Responsiveness:** Allows for near real-time initiation of preprocessing processes immediately following data acquisition [16].

Details regarding integration and code implementation are provided in **Section 9.3 [8](Figure C.8)**.

3.3.4 Container Registry (ACR) — Model Image Repository

Azure Container Registry (ACR) functions as a secure and centralized repository for all containerized variants of trained models, storing Docker images produced through Azure ML. It facilitates smooth deployment to AKS, thereby preventing dependency mismatches [17].

Details regarding custom environment Docker images that are used for trained models are provided in **Section 9.3 (Figure C.6)**.

3.3.5 Azure Kubernetes Service (AKS) — Scalable Model Serving

AKS offers a user-friendly inference serving environment for end-users, ensuring smooth and reliable access to performance.

- **Deployment Strategy:** Containerized models from Azure Container Registry (ACR) are deployed as RESTful API endpoints within Azure Kubernetes Service (AKS) [18][19].
- **Scalability:** AKS supports automatic scaling of pods based on incoming API request load, ensuring consistent response times during peak demand [19][20].
- **Integration Capabilities:** The deployed APIs can be integrated with external applications, including Power BI dashboards and Streamlit-based public portals [20][21].

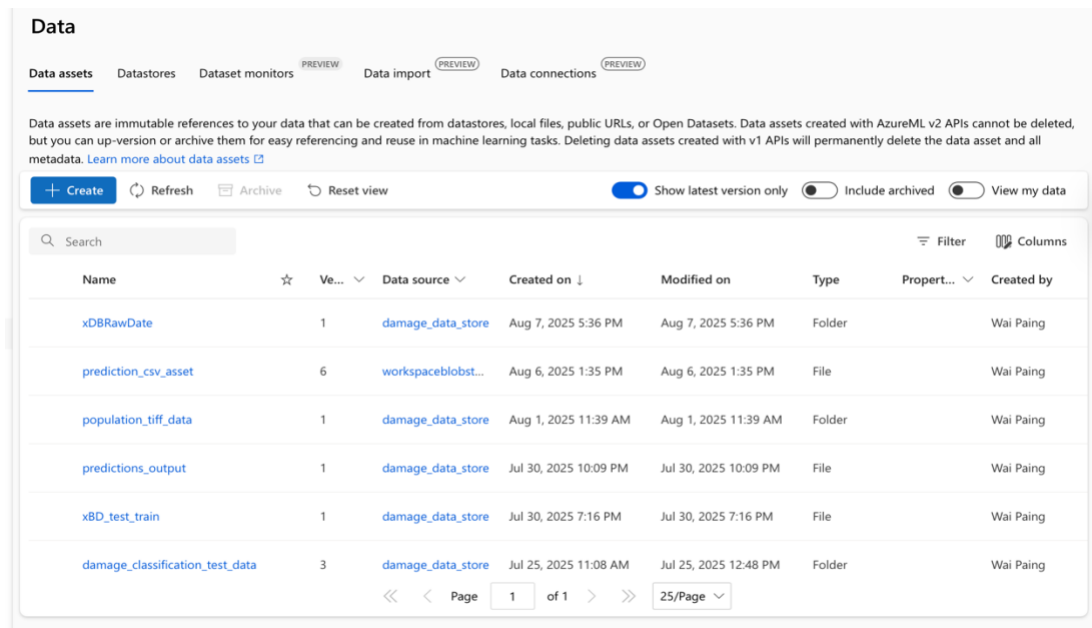
4 RESULTS

4.1 Model Performance

This section assesses the performance of the proposed disaster risk prediction pipeline. It commences with Preprocessing Results, which delineate the preparation of satellite, meteorological, and demographic data for modeling purposes. The Damage Classification Model Results exhibit the accuracy and class-specific metrics of the ResNet50 damage classifier. Subsequently, the Risk Prediction Model Results evaluate the capacity of the XGBoost model to synthesize multiple data sources for post-disaster risk estimation.

4.1.1 Preprocessing Results

The preprocessing phase incorporated raw satellite imagery, meteorological records, and demographic data into standardized datasets for the purposes of model training and evaluation. Several datasets were registered as Azure ML Data Assets to guarantee version control and reproducibility (*Figure 6*).



The screenshot shows the 'Data' section of the Azure ML portal. It includes tabs for 'Data assets', 'Datastores', 'Dataset monitors', 'Data import', and 'Data connections'. Below the tabs is a table of registered data assets. The table has columns for Name, Version, Data source, Created on, Modified on, Type, and Created by. The assets listed are xDBRawDate, prediction_csv_asset, population_tiff_data, predictions_output, xBD_test_train, and damage_classification_test_data. The table also includes a search bar, a filter icon, and a 'Columns' icon. At the bottom, there is a pagination bar showing 'Page 1 of 1' and '25/Page'.

Name	Version	Data source	Created on	Modified on	Type	Created by
xDBRawDate	1	damage_data_store	Aug 7, 2025 5:36 PM	Aug 7, 2025 5:36 PM	Folder	Wai Paing
prediction_csv_asset	6	workspaceblobst...	Aug 6, 2025 1:35 PM	Aug 6, 2025 1:35 PM	File	Wai Paing
population_tiff_data	1	damage_data_store	Aug 1, 2025 11:39 AM	Aug 1, 2025 11:39 AM	Folder	Wai Paing
predictions_output	1	damage_data_store	Jul 30, 2025 10:09 PM	Jul 30, 2025 10:09 PM	File	Wai Paing
xBD_test_train	1	damage_data_store	Jul 30, 2025 7:16 PM	Jul 30, 2025 7:16 PM	File	Wai Paing
damage_classification_test_data	3	damage_data_store	Jul 25, 2025 11:08 AM	Jul 25, 2025 12:48 PM	Folder	Wai Paing

Figure 6: Registered Azure ML Data Assets used in the preprocessing and modelling pipeline.

The Data Asset Summary includes:

- i. xDBRawDate: Raw xBD imagery and its metadata.
- ii. xBD_test_train: Preprocessed image splits for training and testing.
- iii. population_tiff_data: GeoTIFF files showing population density from WorldPop.
- iv. prediction_csv_asset: Model predictions along with metadata (version 6).
- v. predictions_output: Final enriched dataset for dashboards and APIs.
- vi. damage_classification_test_data: Test data used for model evaluation in classification tasks.

All assets are systematically versioned and associated with their respective storage locations, facilitating automated retrieval during pipeline executions. This framework ensures consistent data utilization, straightforward re-executions, and the ability to conduct auditable experiments.

4.1.2 Damage Classification Model Results

The Re-Ensembling (combining ResNet-50, DenseNet, and ElasticNet) classification module classified the damage into four categories: 0 to 3 as No Damage, Minor Damage, Major Damage, and Destroyed.

Performance Summary: The model achieved an accuracy of 93.62% on the training set and 67.52% on the test set.

- Class 0 - No Damage achieved the highest recall at 0.7821 on the test set, demonstrating the model's reliability in detecting intact structures.
- Class 3 - Destroyed also demonstrated strong performance, with a recall of 0.7257, indicating its skill in identifying heavily damaged areas.
- Conversely, Class 1 -Minor Damage showed the weakest performance, with a recall of 0.1404 and a precision of 0.1905. The model was frequently misclassified as Class 0 or Class 2.
- Finally, Class 2 (Major Damage) achieved moderate results, with a balanced precision of 0.5827 and recall of 0.5912.
-

The decline in performance from training to testing indicates overfitting and underscores the difficulty of differentiating between visually similar categories, particularly in overhead satellite imagery.

Class	Precision	Recall	F1-Score
0	0.7102	0.7821	0.7444
1	0.1905	0.1404	0.1616
2	0.5827	0.5912	0.5870
3	0.7600	0.7257	0.7425

Table 1: Test Set Performance

As illustrated in Figure 7, the training confusion matrix exhibits pronounced diagonal dominance, with notably high recall rates for Class 0 and Class 1, and minimal misclassification across most categories.

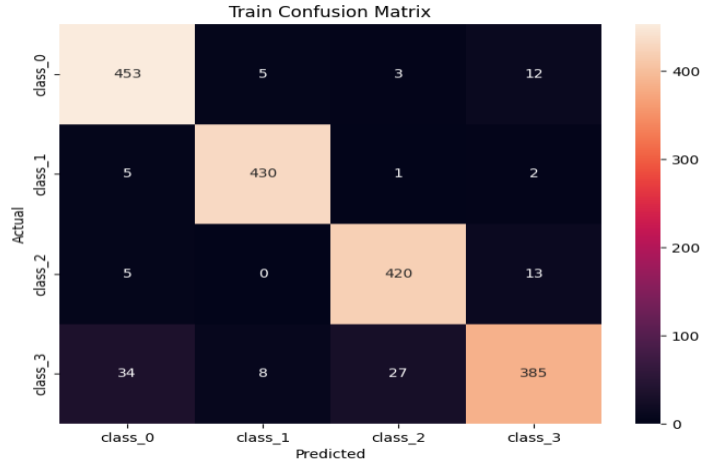


Figure 7: Confusion matrix for the training set showing classification performance across four damage categories.

However, Figure 8 shows that on the test set, Class 1 is notably confused with Class 0 and Class 2, emphasizing the challenge of detecting minor damage from overhead imagery.



Figure 8: Confusion matrix for the test set illustrating model generalization capability.

The ensemble architecture demonstrated effectiveness in identifying non-damaged and heavily destroyed structures; however, it faced challenges with intermediate categories. To enhance performance, considerations may include class balancing, improved data augmentation, and advanced feature extraction techniques capable of capturing subtle structural distinctions. Details of the results log in the Azure Environment are provided in **Section 9.3 [1]**.

4.1.3 Risk Prediction Model Results

The proposed XGBoost-based risk prediction model achieved a training accuracy of 80.4% and a test accuracy of 70.1%, demonstrating satisfactory generalization with moderate overfitting. The training and validation log loss

curves (Figure 1) exhibit a rapid decline within the initial 50 iterations, followed by a gradual decrease until convergence. Validation loss stabilized after approximately 100 iterations, indicating that the model has attained an optimal fit without significant overfitting.

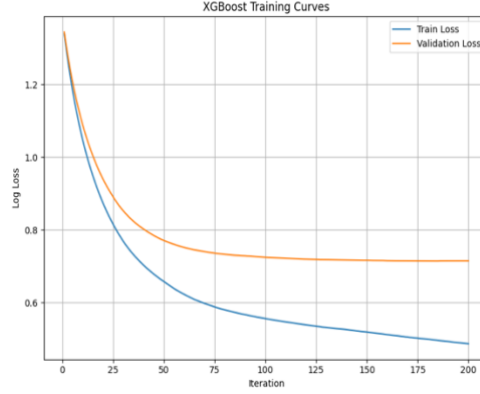


Figure 9: XGBoost training and validation loss curves over 200 iterations.

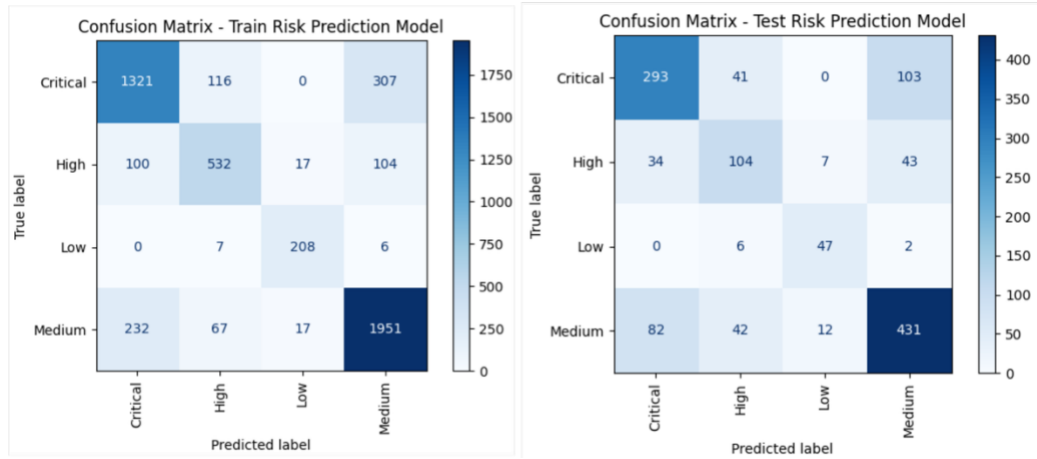


Figure 10: Confusion matrices for the proposed risk prediction model on training and test datasets

Figure 10 illustrates the confusion matrices for the proposed risk prediction model evaluated on both the training and test datasets, encompassing four risk categories: Critical, High, Low, and Medium. In the training dataset, the model demonstrated high recall in the Low category (0.92) and high precision in the Critical (0.85) and Medium (0.88) categories. The test dataset exhibited similar patterns, with the Low category attaining the highest recall (0.85) and the Critical (0.72) and Medium (0.74) categories recording the highest precision.

Misclassification patterns were consistent across both datasets, predominantly occurring between Critical and Medium, and between High and Critical, suggesting partial overlap in their feature distributions.

The classification metrics for the test dataset yielded F1-scores of 0.69 (Critical), 0.55 (High), 0.78 (Low), and 0.75 (Medium). The macro-averaged

and weighted-average F1-scores were 0.69 and 0.70, respectively, indicating stable performance despite mild class imbalance.

4.2 Visualization Output

This section delineates three primary channels for visualization and deployment aimed at interpreting and disseminating the model's results: the Power BI dashboard, API endpoints deployed via Azure Kubernetes Service (AKS), and the Streamlit public web application. The Power BI dashboard facilitates interactive exploration of aggregated predictions, trends, and risk distributions through dynamic filtering and drill-down functionalities. The AKS-hosted API endpoints afford programmatic access to enable seamless integration with external systems. At the same time, the Streamlit application provides a browser-based platform for real-time interaction with the model, suitable for both technical and non-technical users.

4.2.1 Power BI Dashboard

The system includes two types of dashboards: the Response Priority List and the Resource Allocation Map.

1. Response Priority List Dashboard

The Response Priority List dashboard provides an overview and analysis of disaster incidents globally. It was emphasized that the urgency of resource deployment, response speed, and high-risk areas. The dataset covers disasters from 2011 to 2019 across different countries, including six categories: Fire, Earthquake, Flooding, Wind, Tsunami, and Volcano.

Over the nine-year period, key performance indicators such as Critical Locations (2,181) highlight immediate-priority events. An Average Response Time of 6.94 hours reflects the typical time taken to respond after an incident. The Maximum Risk Score of 76.95 stands out as the highest observed composite risk. A global map visually depicts disaster locations, color-coded by immediate resource prioritization. Trend analysis charts monitor annual disaster frequency and average response times. Visualizations of risk distribution display the prevalence of disaster types, with Fire being most frequent, followed by Flooding and Wind, and the composite risk by country, with the highest scores observed in the United States, followed by Portugal, India, Australia, Indonesia, Haiti, Mexico, and Guatemala. Response times are further compared across nations based on priority levels.

This dashboard helps stakeholders by identifying urgent needs, evaluating response effectiveness, and prioritizing high-risk areas.

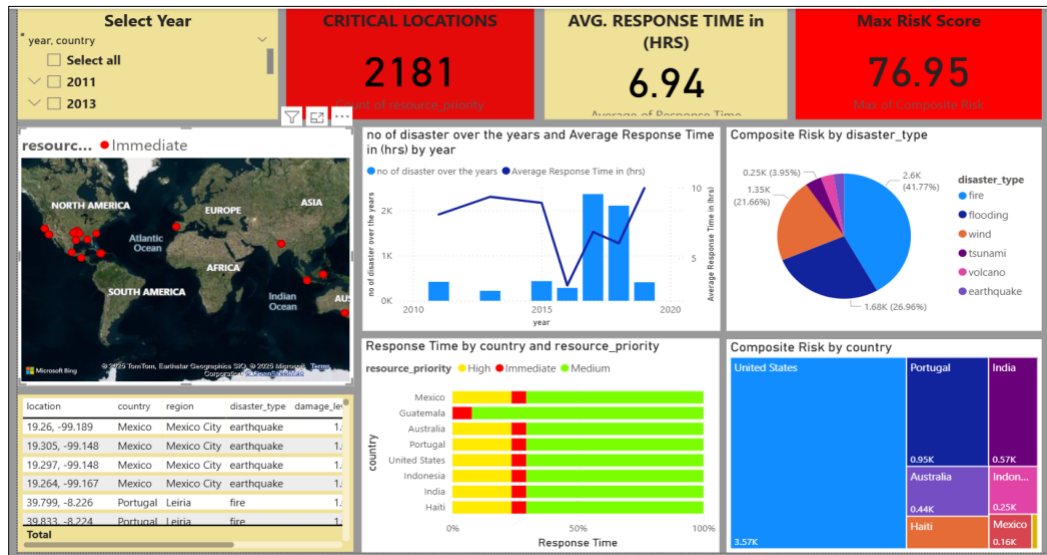


Figure 11: Response Priority List dashboard summarizing global disaster events (2011–2019) by disaster type and country.

2. Resource Allocation Map Dashboard

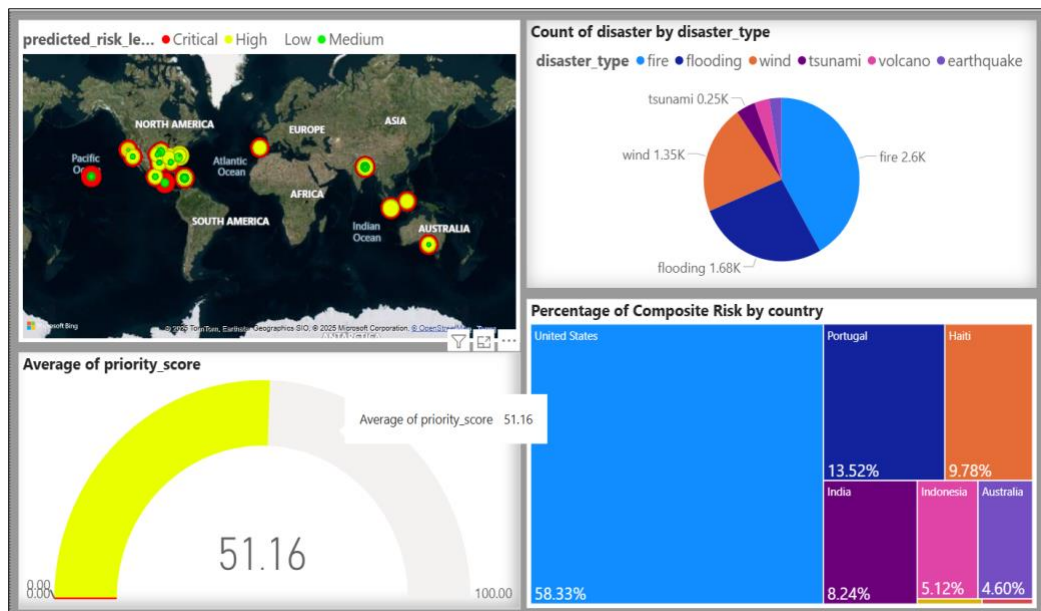


Figure 12: Resource Allocation Map dashboard visualizing global disaster risk distribution (Critical, High, Medium, Low), disaster type frequency, average priority score, and composite risk by country to support data-driven resource deployment.

The Resource Allocation Map dashboard provides a complete overview of how disaster risk is distributed worldwide. It helped prioritize resource deployment based on urgency, risk level, and disaster type. The global map illustrates disaster locations with color codes corresponding to predicted risk categories (Critical, High, Medium, Low). It allows for the swift identification of areas requiring immediate intervention.

A distribution chart of disaster types reveals that Fires are the most prevalent, followed by Flooding and Wind-related incidents. The priority

score gauge indicates an average urgency level of 51.16 out of 100 across all documented events. A composite risk treemap demonstrates that the United States accounts for the largest proportion of total disaster risk (58.33%), succeeded by Portugal, Haiti, India, Indonesia, and Australia.

This dashboard serves as a decision-support tool for global disaster response, combining risk visualization, event classification, and country risk contributions to guide data-driven resource allocation.

4.2.2 API Endpoints (AKS Deployment)

The XGBoost Risk Prediction model is deployed on Azure Kubernetes Service (AKS) as a REST API endpoint:

POST <http://130.107.164.170:80/api/v1/service/xgboost-risk-endpoint/score>

Requests require a Bearer Token and a JSON payload containing disaster-related parameters. The API returns a predicted damage class code, which maps to categories such as No Damage (0), Major Damage (2), and Destroyed (3). A sample request/response format and visualization output are provided in Appendix B.

Note: The above AKS endpoint will remain accessible until the end of August 2025 for course purposes and will be removed afterwards.

4.2.3 Streamlit Public Application

The Streamlit Public App is a web-based interface intended for non-technical users to access the automated damage assessment system. It has been developed to facilitate public accessibility and enable rapid prototyping. It facilitates simple deployment with minimal infrastructure.

1. Purpose and Functionality

Empower end-users, including disaster response teams, non-governmental organizations, and journalists, to interact with the damage assessment system without necessitating technical expertise or Azure access. Provide damage and population risk maps by selecting pre-existing disaster records from the processed dataset.

2. Key Features

a. Geospatial Risk Visualization (Map)

- i. An interactive map with Light, Dark, and Satellite base styles displays disaster events using clustered, color-coded markers: Dark Red (Destroyed), Orange (Major), Yellow (Minor), Green (None).
- ii. A population density heatmap overlay highlights high-risk areas. Pop-ups show disaster type, location, damage level, and population density.

- b. Population Density and Damage Correlation (Risk Matrix)
A heatmap shows the relationship between population density levels (High, Medium, Low) and damage categories, helping identify density–damage trends linked to higher risks.
- c. Weather–Damage Relationship Analysis (Weather Correlation)
A scatter plot visualizes wind speed vs. damage severity, grouped by disaster type. Hovering reveals event details such as name, country, and predicted risk category.
- d. Risk Probability Distribution Analysis (Risk Explorer)
Box plots display probability distributions for four risk categories—Critical, High, Medium, Low—by disaster type and country, enabling a detailed review of prediction patterns.

This public dashboard offers a centralized, interactive platform for monitoring and communicating disaster risk, with the flexibility to integrate real-time data for situational awareness.

3. Integrated Technical Implementation and User Workflow

The Streamlit dashboard, hosted via GitHub, dynamically loads disaster risk data from Azure Blob Storage using a SAS token with caching for efficiency. Data processing utilizes Pandas, while visualizations are rendered across four interactive tabs employing Folium, Seaborn, and Plotly. Application updates are deployed automatically through GitHub.

Users begin by filtering data by disaster type and country, then proceed through the analytical workflow: spatial exploration in the Map tab, exposure-damage analysis in the Risk Matrix, climate impact assessment via Weather-Damage plots, and probabilistic risk evaluation in the Risk Explorer. This pipeline facilitates smooth navigation from broad geographic patterns to detailed risk insights within a unified interface.

4. Sample Output Screenshot

Representative outputs from the Streamlit Public App are shown in Appendix A. These include sample views of the interactive Map, Risk Matrix, Weather Correlation plot, and Risk Explorer probability distribution, demonstrating the dashboard’s main analytical and visualization features.

5. Advantages of the Public App

The Streamlit Public App offers accessible, real-time insights into disaster risks through an interactive web-based interface. It allows users to filter data by disaster type and location for targeted analysis. Its scalable architecture ensures up-to-date info via Azure Blob Storage, and its visualizations aid situational awareness for better decisions.

5 DISCUSSION OF RELATED WORK AND COMPARISON

Disaster damage assessment has historically depended on two primary methods. The initial approach entails manual examination of satellite or aerial imagery, wherein specialists visually analyze structural impairment. Although dependable, this technique is laborious, demands significant resources, and is impractical for large-scale rapid response [7].

The second approach employs machine learning-based remote sensing models, specifically convolutional neural networks (CNNs) such as ResNet and VGG. These models facilitate automated classification of building damage and are capable of processing extensive datasets more efficiently than human experts [8], [9]. However, single-model CNNs frequently encounter limitations in their generalization across various disaster types and face difficulties in differentiating subtle inter-class variations (e.g., “minor” versus “major” damage).

The release of large-scale datasets like xBD [3] has sped up research in automated damage detection. However, most previous studies only focus on structural classification without considering contextual factors such as population density or weather conditions. Baseline CNNs, including ResNet-50, reach around 64–65% accuracy on xBD [3][10] but these models offer little insight for risk prioritization in disaster management efforts.

The proposed system tackles these issues with three main contributions. First, an ensemble model that combines ResNet-50, DenseNet-121, and EfficientNet-B0 raises classification accuracy to 67.5%, beating the 64.82% accuracy of a single ResNet baseline and improving stability for the “No Damage” and “Destroyed” categories. Second, a context-aware risk modeling framework merges CNN outputs with XGBoost, including weather and population data to reach 70.1% accuracy in predicting risks. This allows for evaluating both structural damage and human vulnerability. Third, the system is deployed using a cloud-native Azure ML pipeline, replacing local tests with an automated, scalable process. This pipeline connects Blob Storage, Kubernetes Service, Power BI dashboards, and a public Streamlit app, ensuring scalability and near-real-time decision support.

By enhancing accuracy, adding contextual data, and enabling scalable deployment, the new framework surpasses earlier single-model, classification-only methods, closing the gap between machine learning results and operational disaster response systems.

6 LIMITATIONS AND POSSIBLE EXTENSIONS

While the proposed Azure-based disaster risk prediction pipeline successfully integrates satellite imagery, enriched geospatial datasets, and automated deployment, several limitations remain in both the system design and model performance.

From a system perspective, the current implementation does not include a public-facing application or user interface for uploading new disaster imagery. Data ingestion depends on predefined cloud storage updates, which restricts real-time usage by disaster response teams in the field. In the data enrichment stage, the pipeline relies heavily on public APIs for weather, geospatial, and population datasets; however, it lacks contingency mechanisms for scenarios where such APIs are unavailable or rate-limited.

From a dataset, although the xBD dataset includes both pre- and post-disaster imagery, only post-disaster images were used in this study, reducing the effective training set to 5,594 records. This limits the model's ability to compare pre- and post-event conditions, which is often critical for accurate damage and risk assessment. Additionally, the reduced dataset size may have constrained the diversity of features available for model training.

From a model perspective, baseline classification results (Figures 7 and 8) show strong training performance but notable drops in accuracy on the test set, especially for certain classes. Misclassifications were frequent between class_0 and class_3, as well as between class_2 and other classes, suggesting overlapping feature distributions. When extended with the XGBoost-based risk categorization stage, the model achieved a training accuracy of 80% and a test accuracy of 70%, reflecting moderate overfitting. Class imbalance — particularly in the High risk category — further impacted precision and recall, as shown in the classification report and confusion matrices.

Future work could address these limitations through several directions:

- i. User Interface Development – Implementing a web or mobile platform for direct image uploads would enable field operatives to contribute data in real time.
- ii. Data Redundancy Planning – Incorporating cached datasets, mirrored repositories, or private API integrations would ensure continued operation if public data services failed.
- iii. Dataset Utilization Expansion – Leveraging both pre- and post-disaster images in training, combined with synthetic augmentation (e.g., GAN-generated imagery), would improve model robustness and contextual understanding.
- iv. Model Enhancements – Applying techniques such as SMOTE for class balancing, transfer learning with pre-trained geospatial CNNs, and ensemble architectures combining deep learning and gradient boosting could improve accuracy.
- v. Human-in-the-Loop Validation – Adding expert review stages within Power BI or a dedicated dashboard would allow manual verification of high-stakes predictions before deployment to disaster response teams.

7 CONCLUSIONS

This work introduces an AI-driven disaster assessment and risk prediction pipeline fully deployed on Microsoft Azure. The system effectively combines deep learning-based damage classification with contextual risk modelling, facilitating rapid, automated, and scalable assessments.

Notable achievements include:

- Development of an ensemble CNN architecture. It can achieve good performance for critical categories of damage classification.
- Integration of population and weather datasets into an XGBoost risk model, enhances the prioritization of emergency responses.
- Deployment of a cloud-native pipeline gives reproducibility, automation, and real-time accessibility via dashboards and APIs.

Aligned with the original motivation, the system addresses delays and inefficiencies in traditional assessment processes by cutting down the time needed to produce actionable disaster intelligence from days to hours. While some limitations remain, such as dataset imbalance and dependency on public APIs, the

study shows the feasibility and advantages of an end-to-end Azure-based disaster management system. Future improvements will focus on expanding datasets (including pre- and post-disaster imagery), adding real-time data ingestion, and increasing classification accuracy for intermediate damage levels.

8 REFERENCE

- [1] Aon, Weather, Climate & Catastrophe Insight: 2020 Annual Report, Jan. 2021. <https://aon.mediaroom.com/2021-01-25-Aon-finds-climate-influenced-weather-is-key-driver-of-268B-global-damage-from-2020-natural-disasters-with-64-uninsured>
- [2] C. M. Sullivan, "Humanitarian finance: A missed opportunity to reform before the Summit of the Future," Center for Strategic and International Studies, Apr. 2024. <https://www.csis.org/analysis/humanitarian-finance-missed-opportunity-reform-summit-future>
- [3] R. Gupta et al., "xBD: A Dataset for Assessing Building Damage from Satellite Imagery," arXiv, Nov. 2019. Available: <https://arxiv.org/abs/1911.09296>.arXiv+1
- [4] OpenCage Data Ltd., "OpenCage Geocoding API," OpenCage, 2025. Available: <https://opencagedata.com/api>.opencagedata.com
- [5] NASA Langley Research Center, "NASA Prediction of Worldwide Energy Resources (POWER) API," NASA, 2025. Available: <https://power.larc.nasa.gov/data-access-viewer/>.
- [6] WorldPop, "WorldPop Project," University of Southampton, 2025. Available: <https://www.worldpop.org/>.Wikipedia
- [7] Y. Dong and S. Shan, "A comprehensive review of earthquake-induced building damage detection with remote sensing techniques," ISPRS J. Photogramm. Remote Sens., vol. 150, pp. 245–277, 2019.
- [8] J. Xu, C. Wang, and H. Yang, "Post-disaster building damage assessment from VHR satellite images using transfer learning," Remote Sens., vol. 11, no. 1, p. 113, 2019.
- [9] R. Gupta et al., "Creating xBD: A dataset for assessing building damage from satellite imagery," Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR) Workshops, 2019, pp. 10–17.
- [10] D. Weber and A. Kané, "Building damage assessment with ResNet-based classifiers using xBD dataset," Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS), 2020, pp. 3213–3216.
- [11] Farhangi, A. (2024). Adaptive Anomaly Prediction Models. <https://core.ac.uk/download/645913152.pdf>
- [12] "Challenges to Earthquake Relief Delivery in Myanmar Amid Military Control," Peace Humanity Monitor, Jul. 2025. <https://peacehumanity.org/monitor/challenges-to-earthquake-relief-delivery-in-myanmar-amid-military-control/>.
- [13] K. Ahn et al., "Generalizable disaster damage assessment via change detection," arXiv preprint arXiv:2406.08020, 2024. <https://arxiv.org/html/2406.08020v1>.
- [14] Microsoft, "Secure access and data in workflows - Azure Logic Apps," Microsoft Learn, Nov. 15, 2024. <https://learn.microsoft.com/en-us/azure/logic-apps/logic-apps-securing-a-logic-app>.
- [15] Microsoft, "Trigger Azure Functions on blob containers using an event subscription," Microsoft Learn, Nov. 11, 2024. <https://learn.microsoft.com/en-us/azure/azure-functions/functions-event-grid-blob-trigger>.
- [16] Microsoft, "Azure Blob storage trigger for Azure Functions," Microsoft Learn, May 14, 2025. <https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-storage-blob-trigger>.
- [17] Microsoft, "Azure Container Registry," Microsoft Azure, [Online]. Available: <https://azure.microsoft.com/en-us/products/container-registry>.
- [18] Microsoft, "Azure Container Registry," Microsoft Azure, <https://azure.microsoft.com/en-us/products/container-registry>.
- [19] Microsoft, "Integrate Azure Container Registry with Azure Kubernetes Service," Microsoft Learn, <https://learn.microsoft.com/en-us/azure/aks/cluster-container-registry-integration>.
- [20] Microsoft, "Azure Kubernetes Service (AKS) overview," Microsoft Learn, <https://learn.microsoft.com/en-us/azure/aks/overview>.
- [21] Microsoft, "Power BI REST API overview," Microsoft Learn, <https://learn.microsoft.com/en-us/rest/api/power-bi/>.
- [22] Streamlit Community, "Integrate Streamlit with external dashboards," <https://discuss.streamlit.io/t/powerbi-dashboards-and-streamlit/15323>.

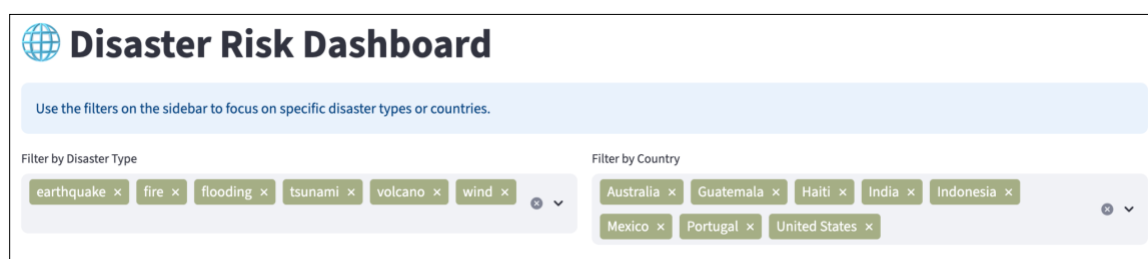
9 APPENDIX

9.1 Appendix A — Sample Output Screenshots for Streamlit Public App

All screenshots in Appendix A were captured from the live application, accessible at:

<https://disaster-dashboard-repo-ppnofytnxwazkjbbdfaanz.streamlit.app>

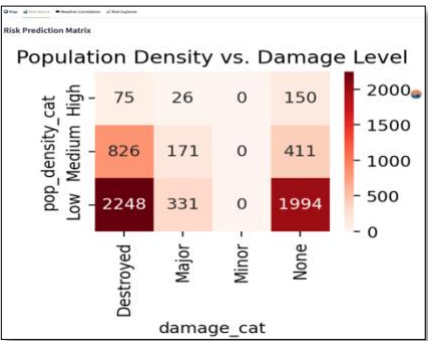
[1] Figure A.1 — Main Filter Panel: Sidebar controls allowing users to filter data by disaster type and country, instantly updating all visualizations.



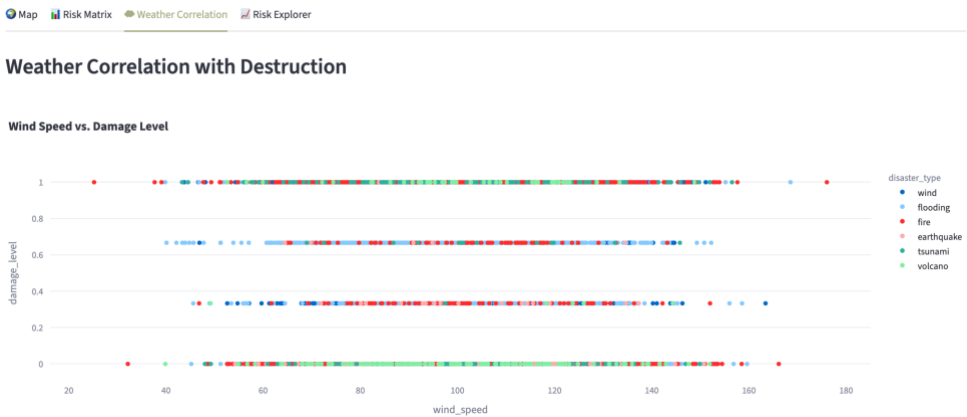
[2] Figure A.2 — Map Tab: Interactive geospatial visualization of disaster events with clustered severity markers and population density heatmap overlay.



[3] **Figure A.3 — Risk Matrix Tab:** Heatmap showing the relationship between population density categories and damage levels.



[4] **Figure A.4 — Weather Correlation Tab:** Scatter plot illustrating the relationship between wind speed and damage severity, color-coded by disaster type.



[5] **Figure A.5 — Risk Explorer Tab:** Box plot displaying predicted probability distributions across four risk categories for a selected disaster type.



Git Repo: <https://github.com/Sarah-Wai/disaster-dashboard-repo>

9.2 Appendix B — Sample Output Screenshots for EndPoint

All screenshots in Appendix B were captured from the postman app , accessible at:

POST <http://130.107.164.170:80/api/v1/service/xgboost-risk-endpoint/score>

XGBoost Risk Prediction

Postman collection to call the XGBoost Risk Prediction Azure endpoint.

POST Predict Disaster Risk

[Open request →](#)

http://130.107.164.170:80/api/v1/service/xgboost-risk-endpoint/score

Damage Class Mapping

Input Sample (Country, Disaster Type)	Predicted Damage Class (Code)	Damage Class Description
Haiti, wind	0	No Damage
United States, flooding	2	Major Damage
United States, fire	3	Destroyed
Mexico, earthquake	3	Destroyed

Authorization Bearer Token

Token <token>

Body raw (json)

```
json
{
  "data": [
    [22.8652, 3.45, 4.31, 18.20435577, -73.77209926, 2, 2016, 3, 28.99],
    [0.0286, 0.52, 1.8, 29.48543178, -95.59340115, 1, 2017, 1, 26.31],
    [35.1465, 0, 1.83, 38.45468694, -122.7362958, 1, 2017, 2, 14.42],
    [21.8674, 3, 1.33, 19.32691275, -99.20496016, 0, 2017, 0, 16.3]
  ]
}
```

```
Body : {
  "data": [
    [22.8652, 3.45, 4.31, 18.20435577, -73.77209926, 2, 2016, 3, 28.99],
    [0.0286, 0.52, 1.8, 29.48543178, -95.59340115, 1, 2017, 1, 26.31],
    [35.1465, 0, 1.83, 38.45468694, -122.7362958, 1, 2017, 2, 14.42],
    [21.8674, 3, 1.33, 19.32691275, -99.20496016, 0, 2017, 0, 16.3]
  ]
}
```

Token : vP7Y04udg0PMQeUUfGi1Sc33TPRxfpAq

Note: The above AKS endpoint will remain accessible until the end of August for course purposes and will be removed afterwards.

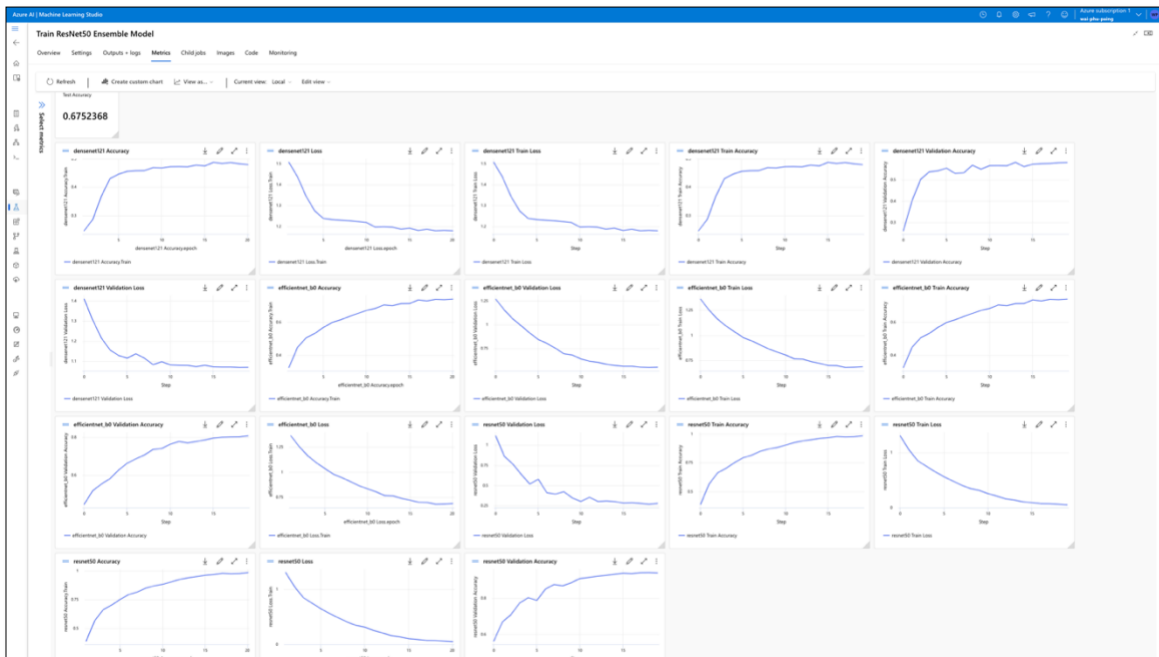
9.3 Appendix C — Implementation, Coding, and Results Documentation

This appendix presents the implementation details, source code repository, and execution outcomes of the proposed system. The GitHub repository link is provided to enable direct access to the complete codebase and configuration files. Additionally, an Azure Machine Learning job run matrix screenshot is included to illustrate the execution workflow and verify the correctness of the deployed pipeline. This documentation ensures reproducibility and transparency in the development process.

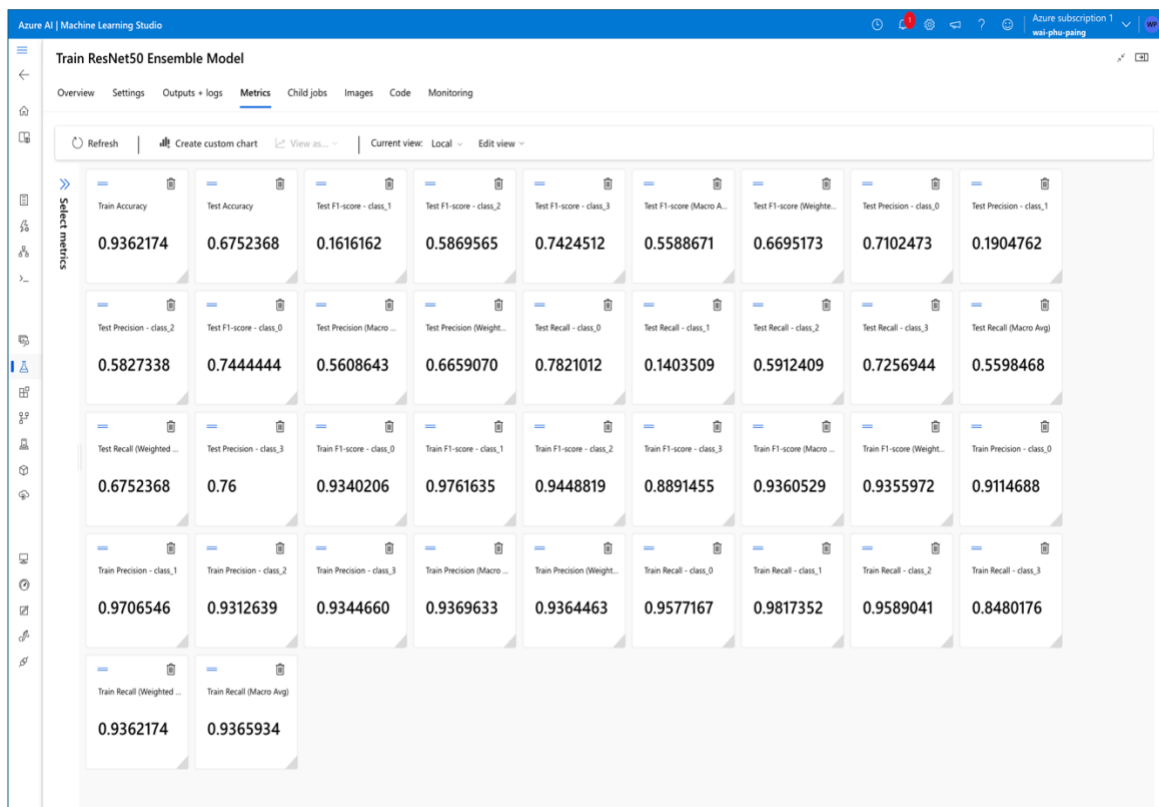
Git Repo : <https://github.com/Sarah-Wai/azure-disaster-managemnet-system>

[1] Figure C.1 (a) & (b)— Azure Machine Learning Metrics Dashboard Displaying Training and Validation Curves for the ResNet50-Based Ensemble Classification Pipeline.

The dashboard depicts the training accuracy, validation accuracy, training loss, and validation loss curves for various base models within the ensemble, including DenseNet121, DenseNet161, EfficientNet-B4, EfficientNet-B0, and ResNet50. Each subplot illustrates a model-specific performance metric plotted against the number of training epochs, facilitating a visual assessment of convergence behavior, overfitting tendencies, and the models' generalization capabilities throughout the development process.



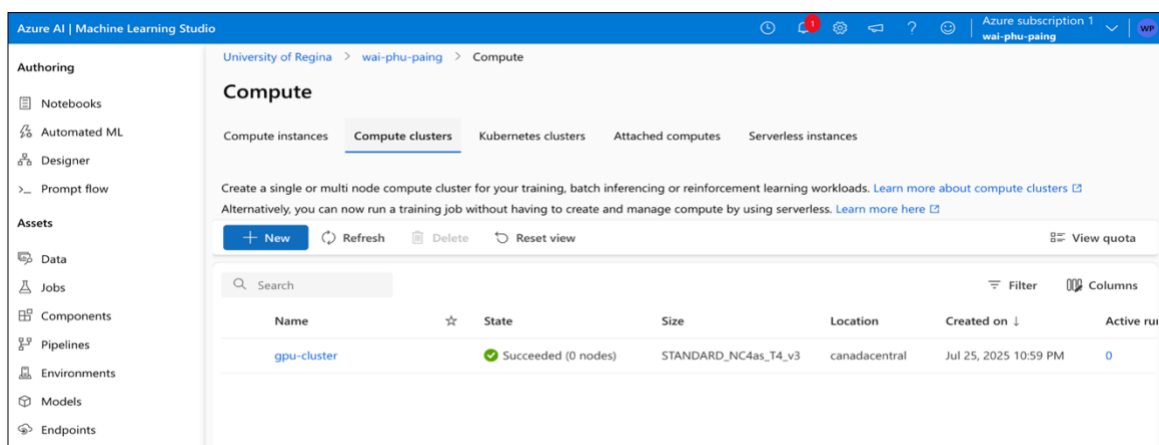
(a) Metric Result 1



(b) Metric Result 2

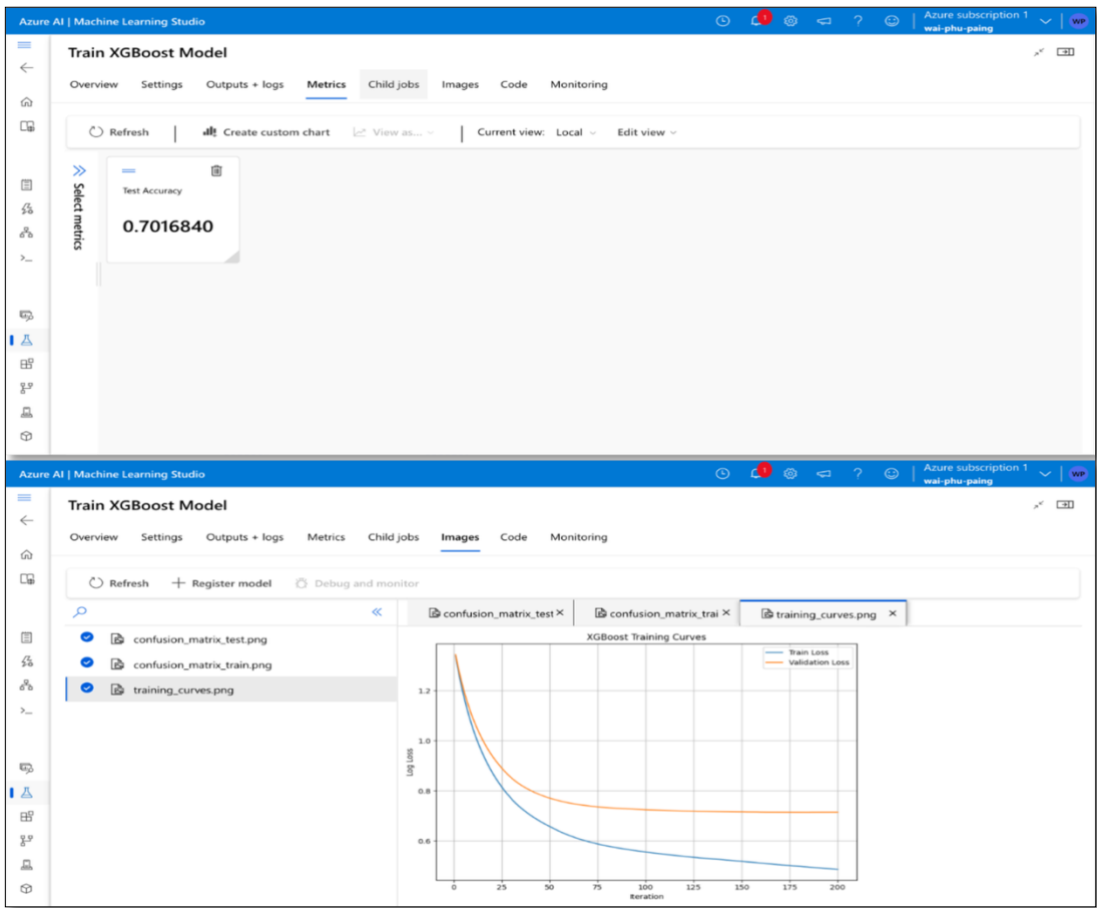
[2] Figure C.2 — Azure Machine Learning Compute Cluster Configuration for Model Training.

The figure illustrates the configuration of the compute cluster within Azure Machine Learning Studio, emphasizing the GPU-enabled cluster (gpu-cluster) provisioned with the STANDARD_NC4as_T4_v3 SKU in the Canada Central region. The cluster is configured for on-demand scaling to facilitate GPU-accelerated model training, batch inferencing, and experimentation activities within the machine learning workflow.



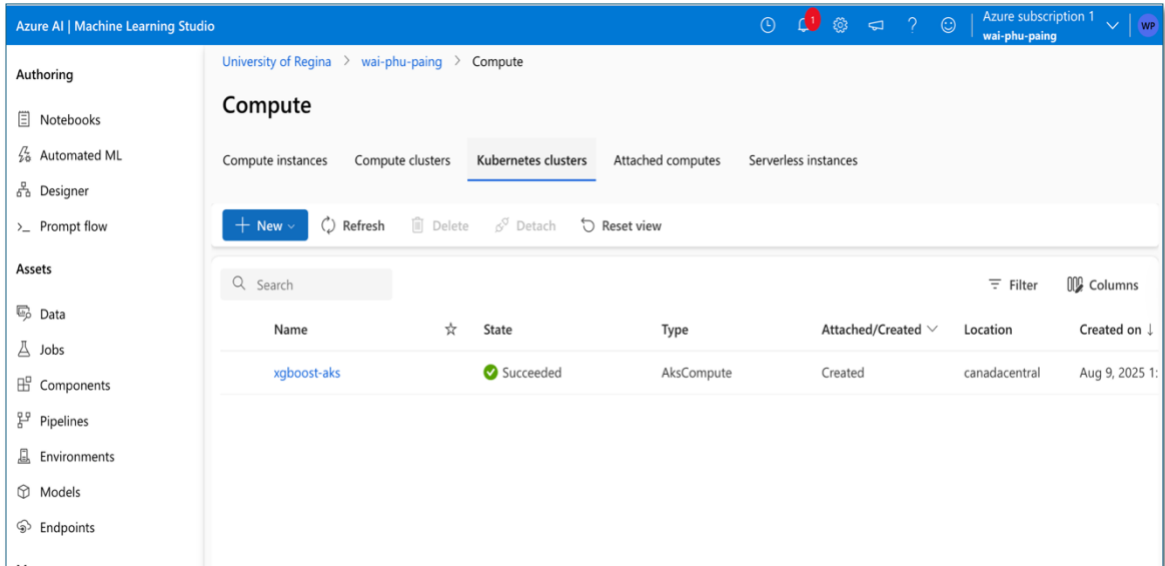
[3] Figure C.3 — Azure Machine Learning Metrics and Visualization Dashboard for the XGBoost Classification Model.

The top panel shows the overall test accuracy metric (0.7016840) from the trained XGBoost model within the classification pipeline. The bottom panel displays the training curves, plotting the logarithmic loss for both the training and validation datasets over 200 boosting iterations. This visualization allows for performance monitoring, early detection of overfitting, and evaluation of convergence during the boosting process.



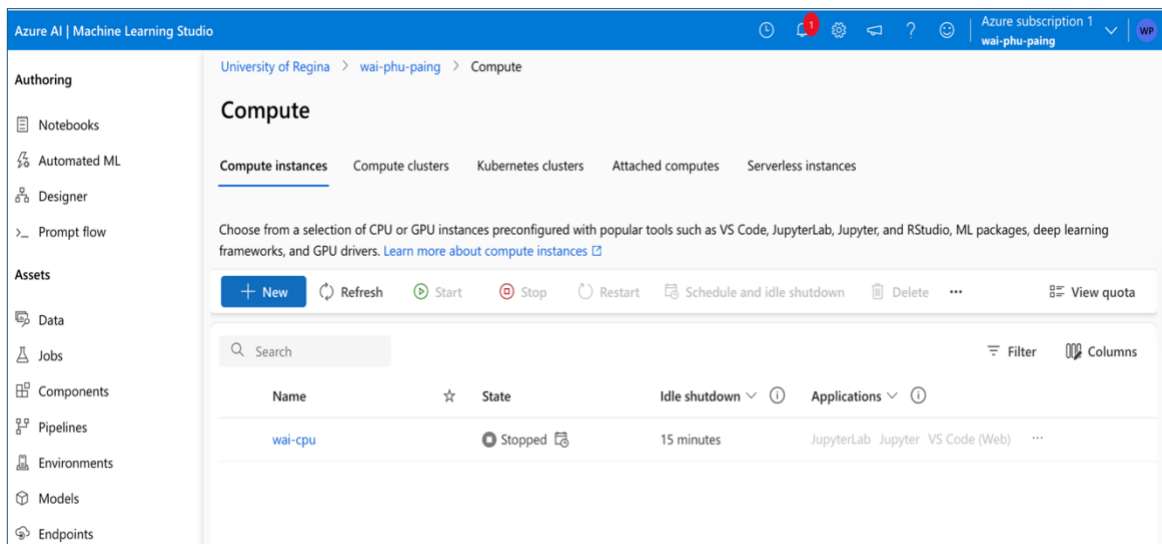
[4] Figure C.4 — Azure Machine Learning AKS Compute Cluster Configuration for Model Training.

The interface displays the AKS compute cluster 'xgboost-aks' (type: AksCompute, status: Succeeded), which has been provisioned in the Canada Central region for scalable model training workflows. This AKS cluster is configured for hosting endpoints.



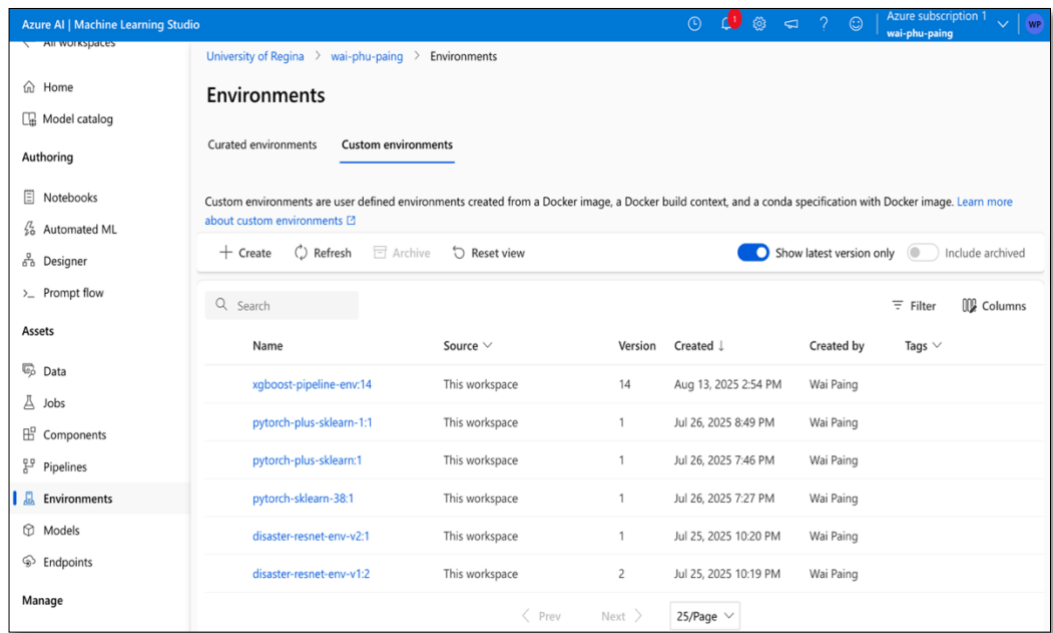
[5] Figure C.5 — Azure Machine Learning CPU Compute Cluster Configuration for Model Training.

The CPU instance "wai-cpu" is designated for all jobs that do not require a GPU. All pipelines will begin execution on this CPU instance by default.



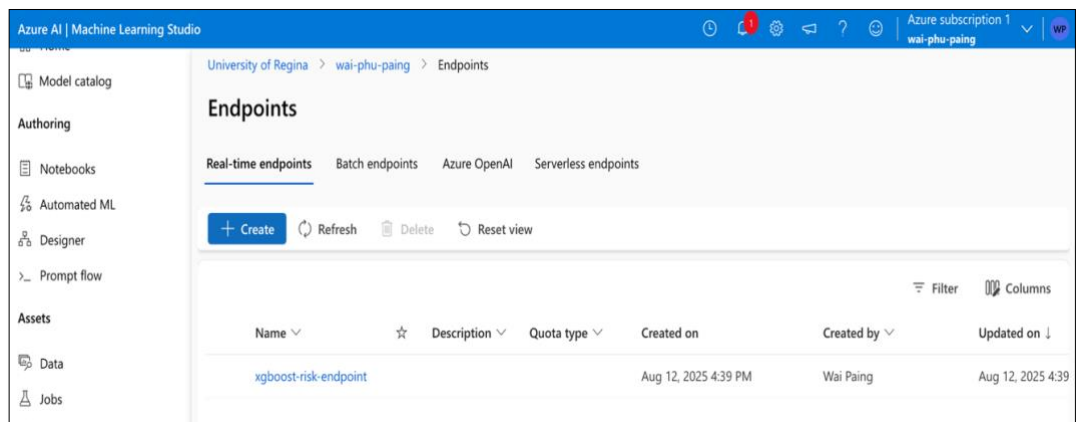
[6] Figure C.6 — Azure ML custom environment configuration interface

This figure illustrates user-defined environments (e.g., xgboost-pipeline-env, pytorch-plus-sklearn) crafted to address dependency conflicts, guarantee experimental reproducibility, and support specialized machine learning workflows. These custom environments offer precise control over Docker images, Conda configurations, and library versions, surpassing the functionalities of curated environments.



[7] Figure C.7 — Azure ML endpoint management interface

This figure illustrates the real-time and batch endpoint, xgboost-risk-endpoint, utilized for deploying machine learning models. The interface facilitates the creation, monitoring, and administration of endpoints, thereby enabling the operationalization of models as scalable services.



[8] Figure C.8 — Azure ML Function Automation

This figure illustrates the integration of function and code to call preprocessing pipeline job.

The figure consists of two screenshots from the Microsoft Azure portal, illustrating the configuration of an Azure ML Function Automation.

Top Screenshot: BlobTrigger | Code + Test

The top screenshot shows the 'Code + Test' tab for the 'BlobTrigger' function. The code is written in JavaScript and is as follows:

```
1 const axios = require("axios");
2 const { DefaultAzureCredential } = require("@azure/identity");
3
4 module.exports = async function (context, myBlob) {
5   context.log('Blob trigger fired for blob: %s(context.bindingData.name)');
6
7   const credential = new DefaultAzureCredential();
8   const token = await credential.getToken("https://management.azure.com/default");
9
10
11   const subscriptionId = "12345678901234567890";
12   const resourceGroup = "myResourceGroup";
13   const workspaceName = "myWorkspace";
14   const jobId = "myJobId";
15
16   const url = `https://management.azure.com/subscriptions/${subscriptionId}/resourceGroups/${resourceGroup}/providers/Microsoft.MachineLearningServices/workspaces/${workspaceName}/jobs/${jobId}`;
17
18   try {
19     const response = await axios.post(url, {}, {
20       headers: {
21         Authorization: `Bearer ${token.token}`,
22         "Content-Type": "application/json"
23       }
24     });
25   } catch (error) {
26     context.log(error);
27   }
28 }
```

Bottom Screenshot: BlobTrigger | Integration

The bottom screenshot shows the 'Integration' tab for the 'BlobTrigger' function. The integration diagram shows the following components:

- Trigger and inputs:** Trigger: Azure Blob Storage (myBlob). Inputs: No inputs defined.
- Function:** BlobTrigger.
- Outputs:** No outputs defined.

The diagram shows a flow from the Trigger and inputs section to the Function section, and then to the Outputs section.